



```
In [19]: import numpy as np
import pandas as pd
df = pd.read_csv("student-mat.csv", sep=';')
print(df.head())
print(df.columns)
```

```

  school sex  age address famsize Pstatus  Medu  Fedu    Mjob    Fjob  ...
\
0      GP   F   18      U    GT3      A     4     4  at_home  teacher  ...
1      GP   F   17      U    GT3      T     1     1  at_home   other  ...
2      GP   F   15      U    LE3      T     1     1  at_home   other  ...
3      GP   F   15      U    GT3      T     4     2  health  services  ...
4      GP   F   16      U    GT3      T     3     3   other    other  ...

```

```

  famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0      4         3      4     1     1     3         6   5   6   6
1      5         3      3     1     1     3         4   5   5   6
2      4         3      2     2     3     3        10   7   8  10
3      3         2      2     1     1     5         2  15  14  15
4      4         3      2     1     2     5         4   6  10  10

```

[5 rows x 33 columns]

```
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
      'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',
      'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
      'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc',
      'Walc', 'health', 'absences', 'G1', 'G2', 'G3'],
      dtype='object')
```

```
In [20]: df.head()
df.info()
df.describe()
df.isnull().sum()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   school                 395 non-null   object
1   sex                   395 non-null   object
2   age                   395 non-null   int64
3   address               395 non-null   object
4   famsize               395 non-null   object
5   Pstatus               395 non-null   object
6   Medu                  395 non-null   int64
7   Fedu                  395 non-null   int64
8   Mjob                  395 non-null   object
9   Fjob                  395 non-null   object
10  reason                395 non-null   object
11  guardian              395 non-null   object
12  traveltime            395 non-null   int64
13  studytime             395 non-null   int64
14  failures              395 non-null   int64
15  schoolsup             395 non-null   object
16  famsup                395 non-null   object
17  paid                  395 non-null   object
18  activities            395 non-null   object
19  nursery              395 non-null   object
20  higher                395 non-null   object
21  internet              395 non-null   object
22  romantic              395 non-null   object
23  famrel                395 non-null   int64
24  freetime              395 non-null   int64
25  goout                 395 non-null   int64
26  Dalc                  395 non-null   int64
27  Walc                  395 non-null   int64
28  health                395 non-null   int64
29  absences              395 non-null   int64
30  G1                    395 non-null   int64
31  G2                    395 non-null   int64
32  G3                    395 non-null   int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB

```

```
Out[20]: school      0
sex                0
age                0
address            0
famsize            0
Pstatus            0
Medu               0
Fedu               0
Mjob               0
Fjob               0
reason             0
guardian            0
traveltime         0
studytime          0
failures           0
schoolsup           0
famsup             0
paid               0
activities          0
nursery            0
higher             0
internet           0
romantic           0
famrel             0
freetime           0
goout              0
Dalc               0
Walc               0
health             0
absences           0
G1                 0
G2                 0
G3                 0
dtype: int64
```

```
In [38]: def classify_grade(g):
        if g < 10:
            return "Fail"
        elif g < 14:
            return "Average"
        else:
            return "Excellent"

        df['Performance'] = df['G3'].apply(classify_grade)
```

```
In [40]: print(df['Performance'].value_counts())
```

```
Performance
Average      165
Fail          130
Excellent    100
Name: count, dtype: int64
```

```
In [74]: X = df.drop(['G3', 'Performance'], axis=1)
y = df['Performance']
print("\nFEATURE MATRIX SHAPE:", X.shape)
print("TARGET VECTOR SHAPE:", y.shape)
print(X.columns)
```

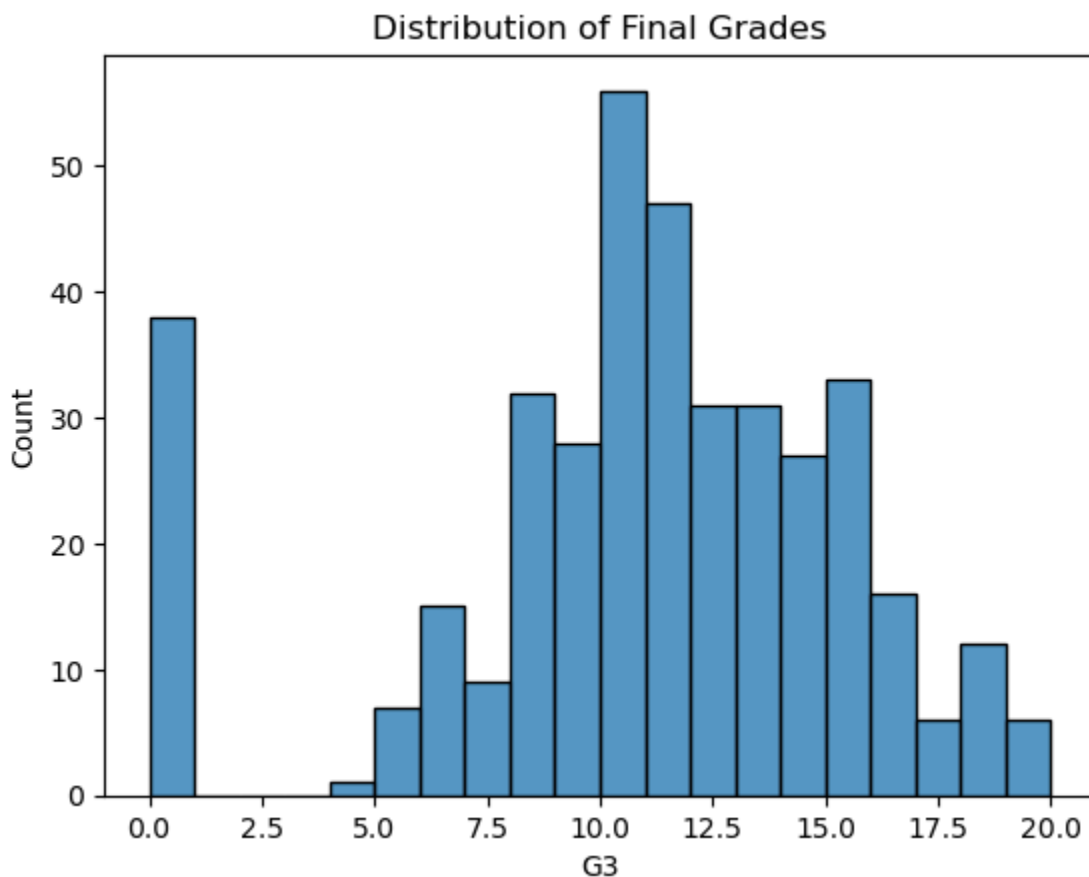
FEATURE MATRIX SHAPE: (395, 32)

TARGET VECTOR SHAPE: (395,)

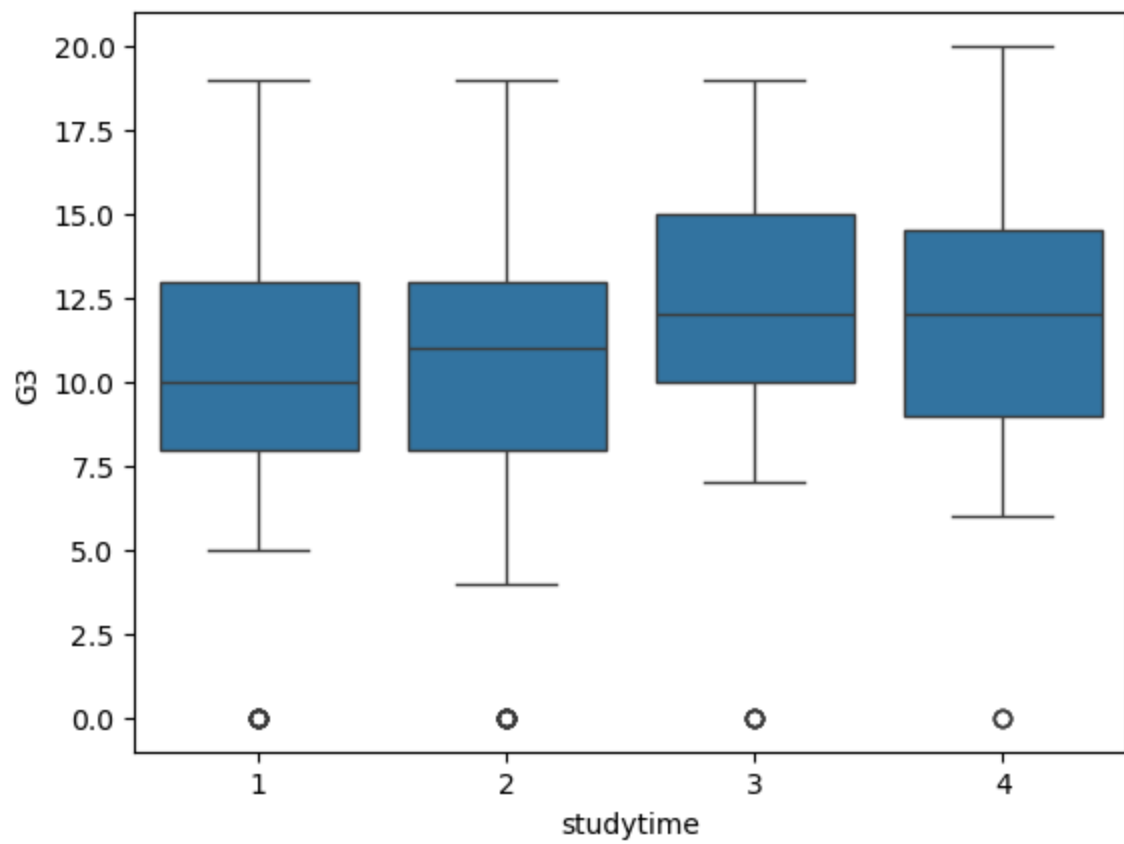
```
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
      'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',
      'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
      'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc',
      'Walc', 'health', 'absences', 'G1', 'G2'],
      dtype='object')
```

```
In [75]: import seaborn as sns
import matplotlib.pyplot as plt

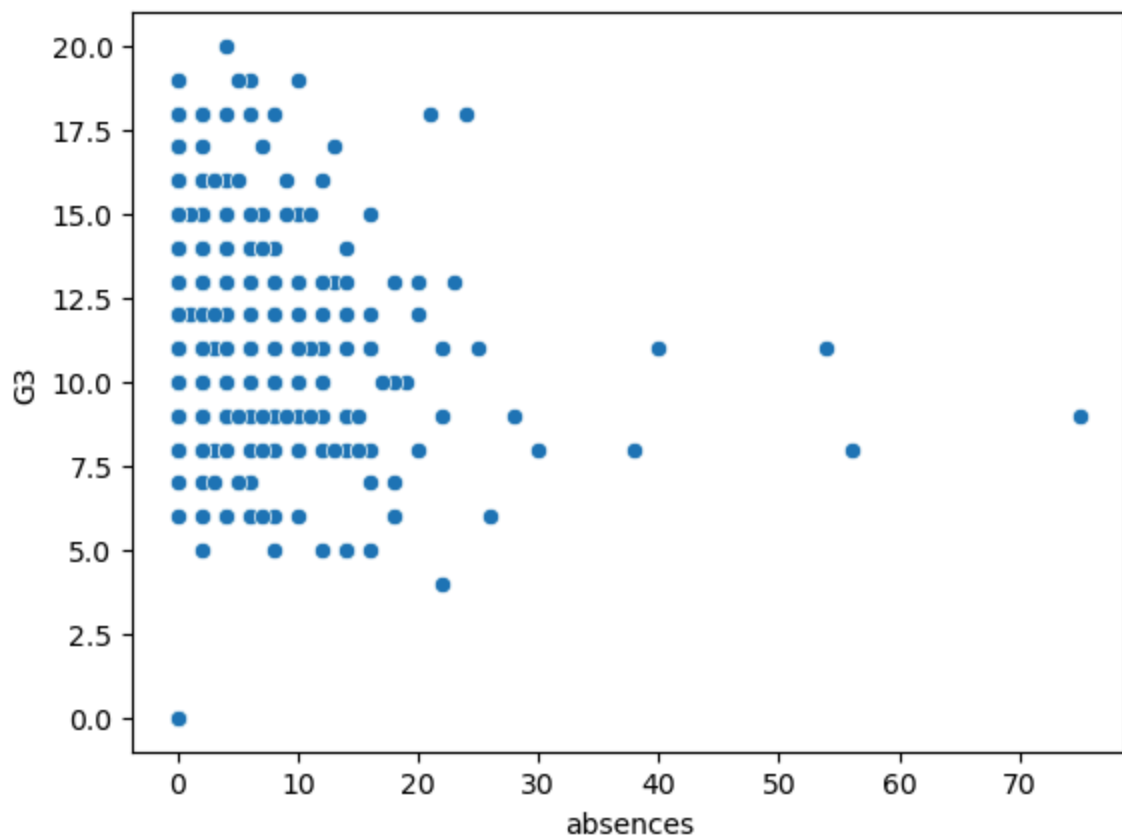
sns.histplot(df['G3'], bins=20)
plt.title("Distribution of Final Grades")
plt.show()
```



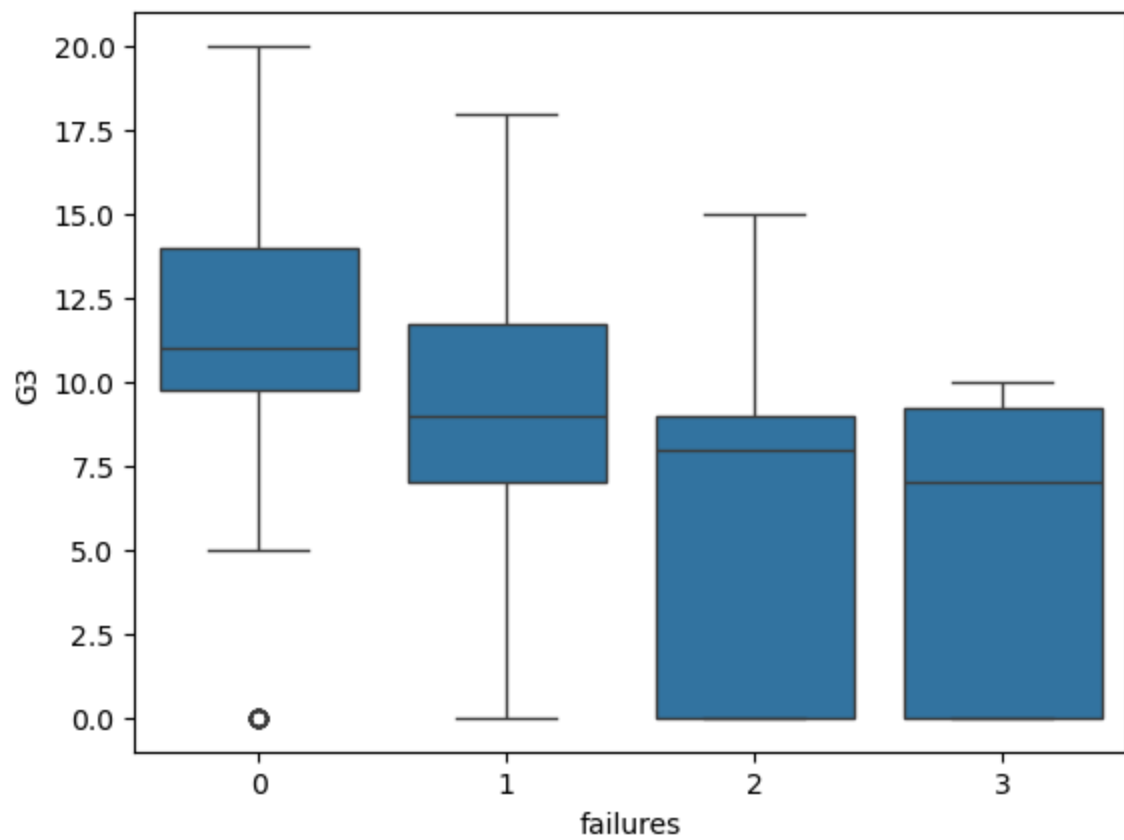
```
In [76]: sns.boxplot(x='studytime', y='G3', data=df)
plt.show()
```



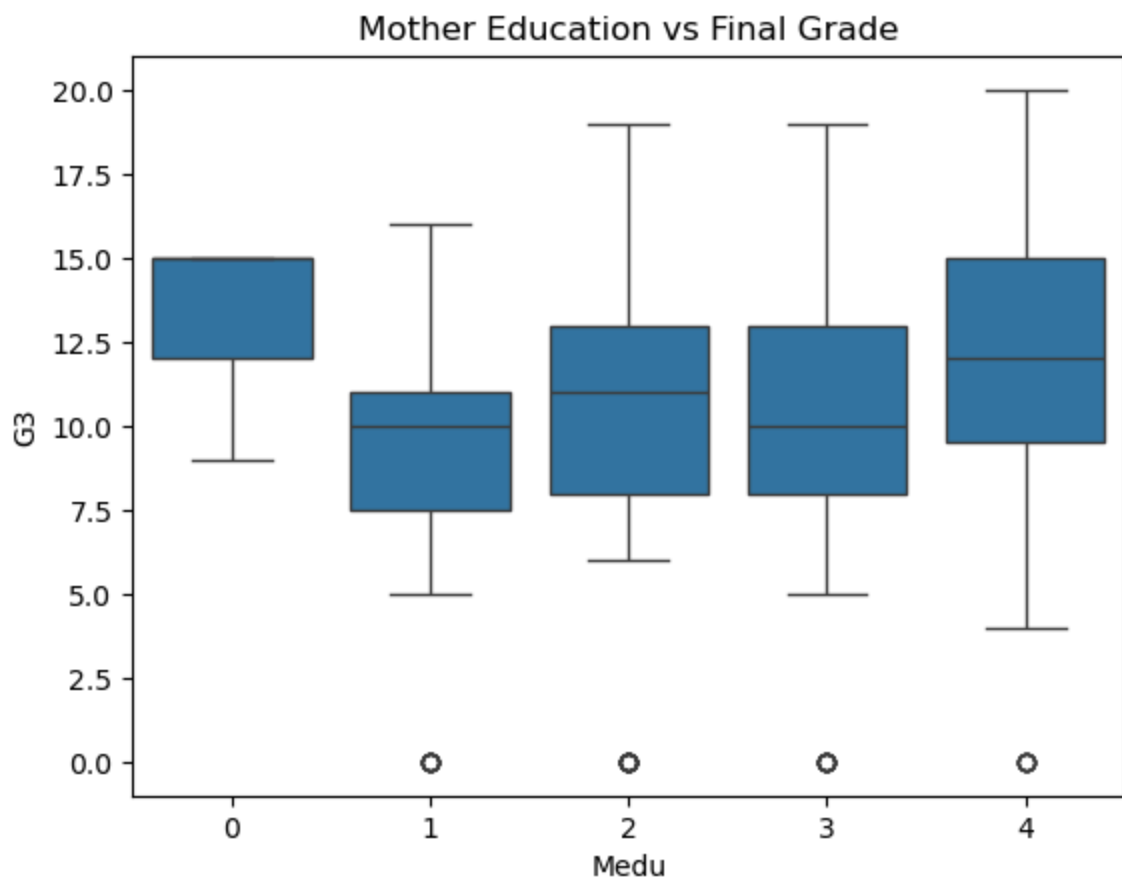
```
In [77]: sns.scatterplot(x='absences', y='G3', data=df)
plt.show()
```



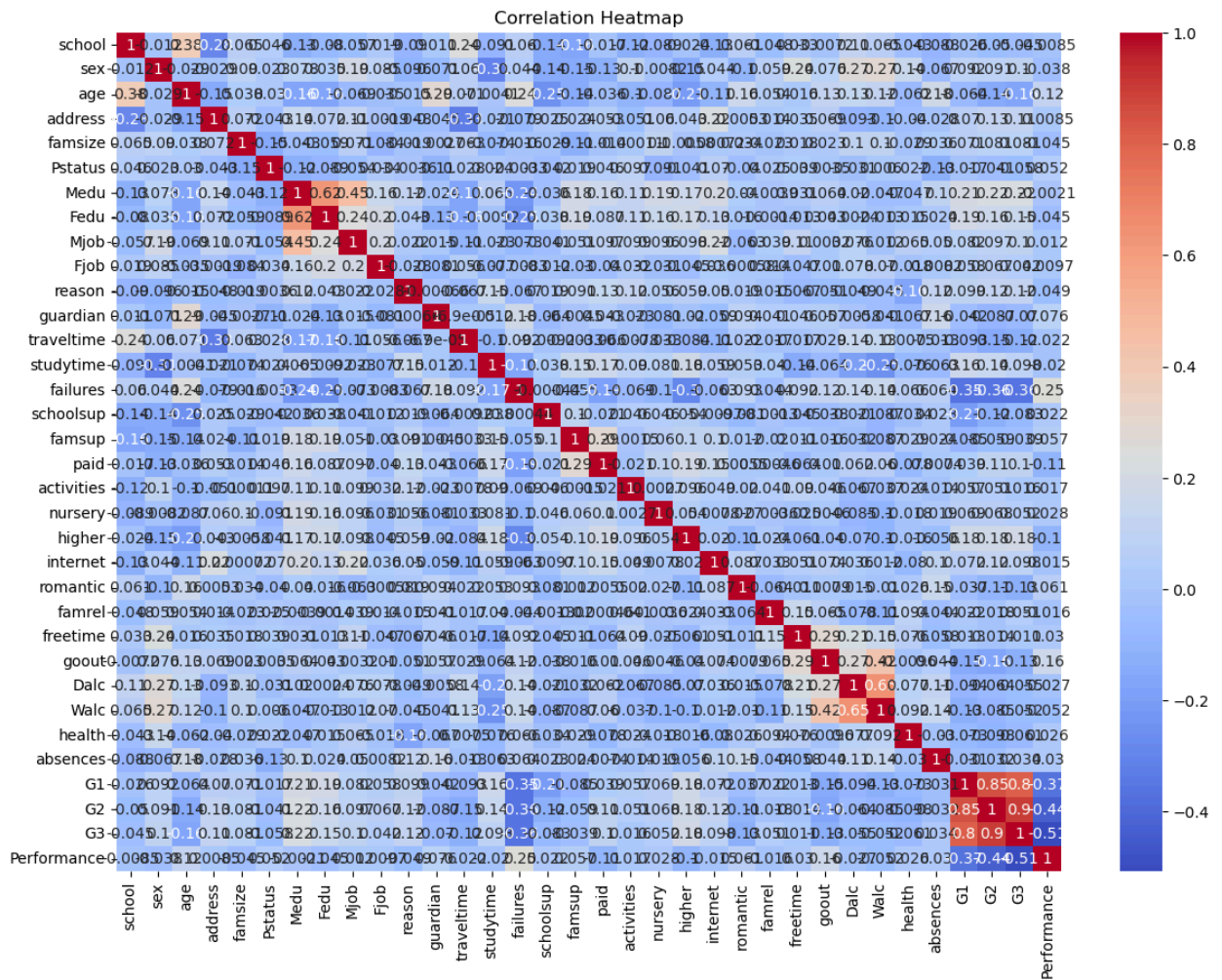
```
In [78]: sns.boxplot(x='failures', y='G3', data=df)
plt.show()
```



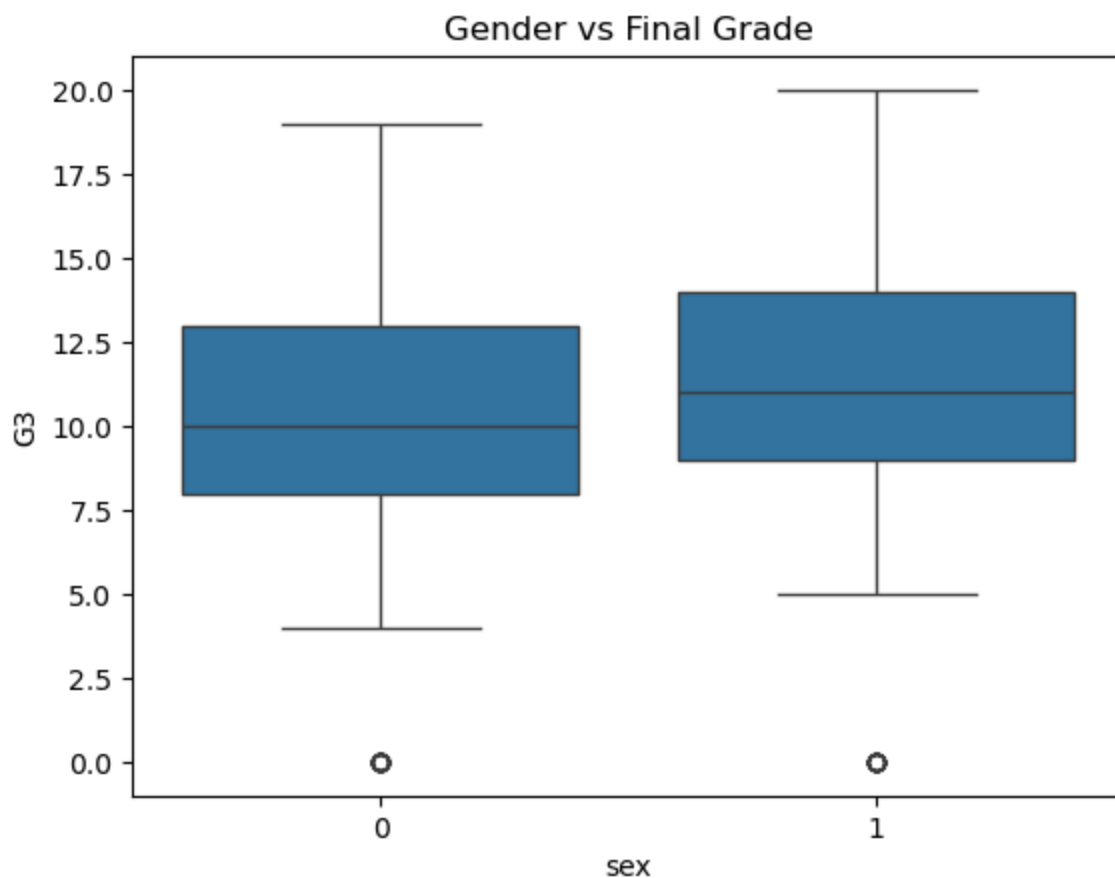
```
In [88]: sns.boxplot(x='Medu', y='G3', data=df)
plt.title("Mother Education vs Final Grade")
plt.show()
```



```
In [91]: numeric_df = df.select_dtypes(include=['int64'])
plt.figure(figsize=(14,10))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

```
In [89]: sns.boxplot(x='sex', y='G3', data=df)
plt.title("Gender vs Final Grade")
plt.show()
```



In [105]: `from sklearn.preprocessing import LabelEncoder`

`le = LabelEncoder()`

```
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
print(df.head())
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	\
0	0	0	18	1	0	0	4	4	0	4	...	
1	0	0	17	1	0	1	1	1	0	2	...	
2	0	0	15	1	1	1	1	1	0	2	...	
3	0	0	15	1	0	1	4	2	1	3	...	
4	0	0	16	1	0	1	3	3	2	2	...	

	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3	Performance
0	3	4	1	1	3	6	5	6	6	2
1	3	3	1	1	3	4	5	5	6	2
2	3	2	2	3	3	10	7	8	10	0
3	2	2	1	1	5	2	15	14	15	1
4	3	2	1	2	5	4	6	10	10	0

[5 rows x 34 columns]

In [92]: `selected_features = [`

```

'school',
'sex',
'age',
'address',
'famsize',
'Pstatus',
'Medu',
'Fedu',
'Mjob',
'Fjob',
'guardian',
'studytime',
'failures',
'schoolsup',
'famsup',
'paid',
'higher',
'internet',
'absences',
'G1',
'G2'
]
X = df[selected_features]

y = df['Performance']

print("\nFEATURES:\n")
print(X.columns)

```

FEATURES:

```

Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
      'Mjob', 'Fjob', 'guardian', 'studytime', 'failures', 'schoolsup',
      'famsup', 'paid', 'higher', 'internet', 'absences', 'G1', 'G2'],
      dtype='object')

```

```

In [93]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_s

```

```

In [94]: from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train_Scaled=scaler.fit_transform(X_train)
X_test_Scaled=scaler.transform(X_test)

```

```

In [95]: from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=2000,class_weight='balanced')
lr.fit(X_train_Scaled, y_train)
lr_pred=lr.predict(X_test_Scaled)
lr_accuracy = accuracy_score(y_test, lr_pred)
print("LOGISTIC REGRESSION RESULTS")
print("\nAccuracy :", lr_accuracy)

```

```

print("\nClassification Report:\n")
print(classification_report(y_test, lr_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, lr_pred))

```

LOGISTIC REGRESSION RESULTS

Accuracy : 0.8481012658227848

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.73	0.76	26
1	0.92	0.88	0.90	26
2	0.83	0.93	0.88	27
accuracy			0.85	79
macro avg	0.85	0.85	0.85	79
weighted avg	0.85	0.85	0.85	79

Confusion Matrix:

```

[[19  2  5]
 [ 3 23  0]
 [ 2  0 25]]

```

```

In [96]: from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=500, random_state=42, max_depth=10, clas
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
print("RANDOM FOREST RESULTS")
print("\nAccuracy :", rf_accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, rf_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, rf_pred))

```

RANDOM FOREST RESULTS

Accuracy : 0.8481012658227848

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.77	0.77	26
1	0.96	0.85	0.90	26
2	0.83	0.93	0.88	27
accuracy			0.85	79
macro avg	0.85	0.85	0.85	79
weighted avg	0.85	0.85	0.85	79

Confusion Matrix:

```
[[20  1  5]
 [ 4 22  0]
 [ 2  0 25]]
```

```
In [97]: importance = pd.DataFrame({
        'Feature': X.columns,
        'Importance': rf.feature_importances_
    })

importance = importance.sort_values(
    by='Importance',
    ascending=False
)
print("FEATURE IMPORTANCE")
print(importance)
```

FEATURE IMPORTANCE

	Feature	Importance
20	G2	0.436198
19	G1	0.229444
18	absences	0.048069
12	failures	0.030857
2	age	0.030527
6	Medu	0.027407
11	studytime	0.026236
7	Fedu	0.022303
8	Mjob	0.022270
9	Fjob	0.020834
13	schoolsup	0.014993
10	guardian	0.013797
15	paid	0.012779
4	famsize	0.011079
1	sex	0.010331
17	internet	0.009583
3	address	0.007539
14	famsup	0.007454
5	Pstatus	0.007136
16	higher	0.006439
0	school	0.004723



```
In [1]: import numpy as np
import pandas as pd
df = pd.read_csv("student-por.csv", sep=';')
print(df.head())
print(df.columns)
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...
\											
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...
1	GP	F	17	U	GT3	T	1	1	at_home	other	...
2	GP	F	15	U	LE3	T	1	1	at_home	other	...
3	GP	F	15	U	GT3	T	4	2	health	services	...
4	GP	F	16	U	GT3	T	3	3	other	other	...

	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	4	3	4	1	1	3	4	0	11	11
1	5	3	3	1	1	3	2	9	11	11
2	4	3	2	2	3	3	6	12	13	12
3	3	2	2	1	1	5	0	14	14	14
4	4	3	2	1	2	5	0	11	13	13

[5 rows x 33 columns]

```
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
      'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',
      'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
      'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc',
      'Walc', 'health', 'absences', 'G1', 'G2', 'G3'],
      dtype='object')
```

```
In [2]: df.head()
df.info()
df.describe()
df.isnull().sum()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 649 entries, 0 to 648
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   school                 649 non-null    object
1   sex                   649 non-null    object
2   age                   649 non-null    int64
3   address               649 non-null    object
4   famsize               649 non-null    object
5   Pstatus              649 non-null    object
6   Medu                 649 non-null    int64
7   Fedu                 649 non-null    int64
8   Mjob                 649 non-null    object
9   Fjob                 649 non-null    object
10  reason               649 non-null    object
11  guardian             649 non-null    object
12  traveltime           649 non-null    int64
13  studytime            649 non-null    int64
14  failures             649 non-null    int64
15  schoolsup            649 non-null    object
16  famsup              649 non-null    object
17  paid                 649 non-null    object
18  activities           649 non-null    object
19  nursery             649 non-null    object
20  higher              649 non-null    object
21  internet            649 non-null    object
22  romantic             649 non-null    object
23  famrel              649 non-null    int64
24  freetime            649 non-null    int64
25  goout               649 non-null    int64
26  Dalc                649 non-null    int64
27  Walc                649 non-null    int64
28  health              649 non-null    int64
29  absences            649 non-null    int64
30  G1                  649 non-null    int64
31  G2                  649 non-null    int64
32  G3                  649 non-null    int64
dtypes: int64(16), object(17)
memory usage: 167.4+ KB

```



```
Out[2]: school      0
sex              0
age              0
address          0
famsize          0
Pstatus          0
Medu             0
Fedu             0
Mjob             0
Fjob             0
reason           0
guardian         0
traveltime       0
studytime        0
failures         0
schoolsup        0
famsup           0
paid             0
activities       0
nursery          0
higher           0
internet         0
romantic         0
famrel           0
freetime         0
goout            0
Dalc             0
Walc             0
health           0
absences         0
G1               0
G2               0
G3               0
dtype: int64
```

```
In [3]: def classify_grade(g):
        if g < 10:
            return "Fail"
        elif g < 14:
            return "Average"
        else:
            return "Excellent"

        df['Performance'] = df['G3'].apply(classify_grade)
```

```
In [4]: print(df['Performance'].value_counts())
```

```
Performance
Average      355
Excellent    194
Fail          100
Name: count, dtype: int64
```

```
In [5]: X = df.drop(['G3', 'Performance'], axis=1)
y = df['Performance']
print("\nFEATURE MATRIX SHAPE:", X.shape)
print("TARGET VECTOR SHAPE:", y.shape)
print(X.columns)
```

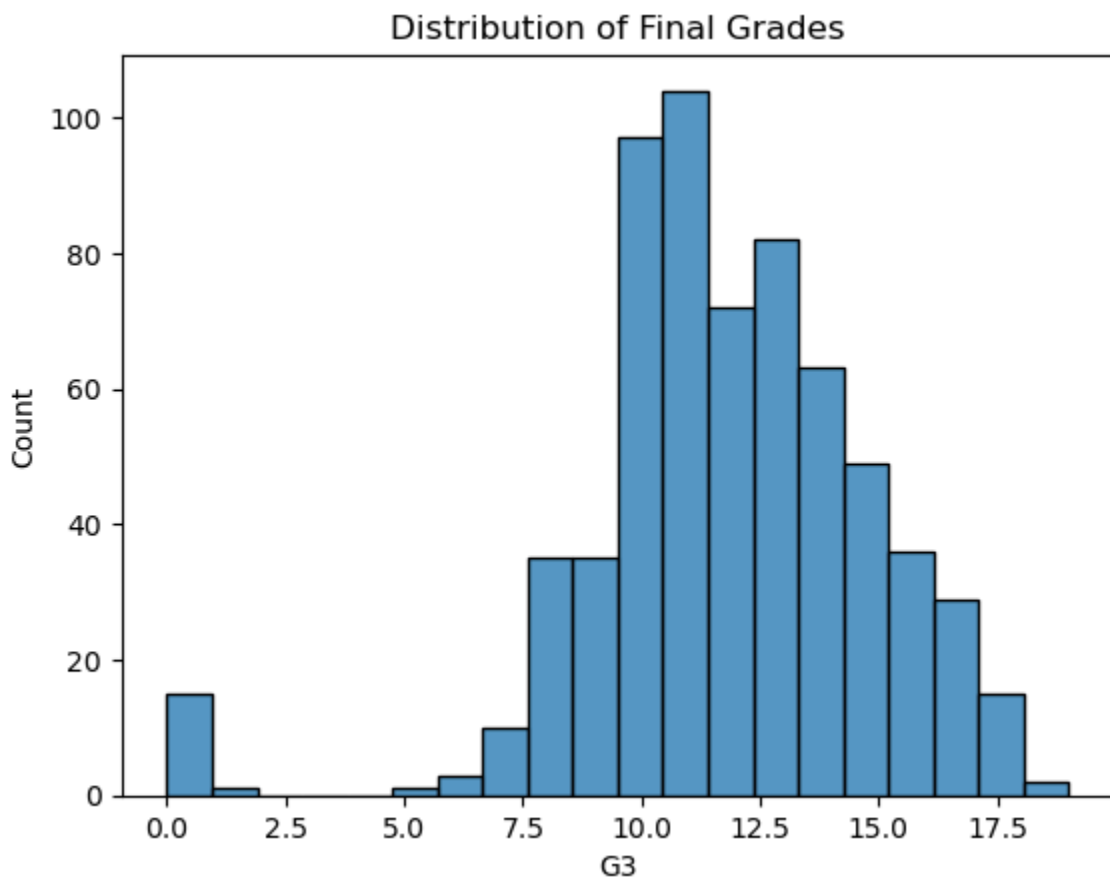
FEATURE MATRIX SHAPE: (649, 32)

TARGET VECTOR SHAPE: (649,)

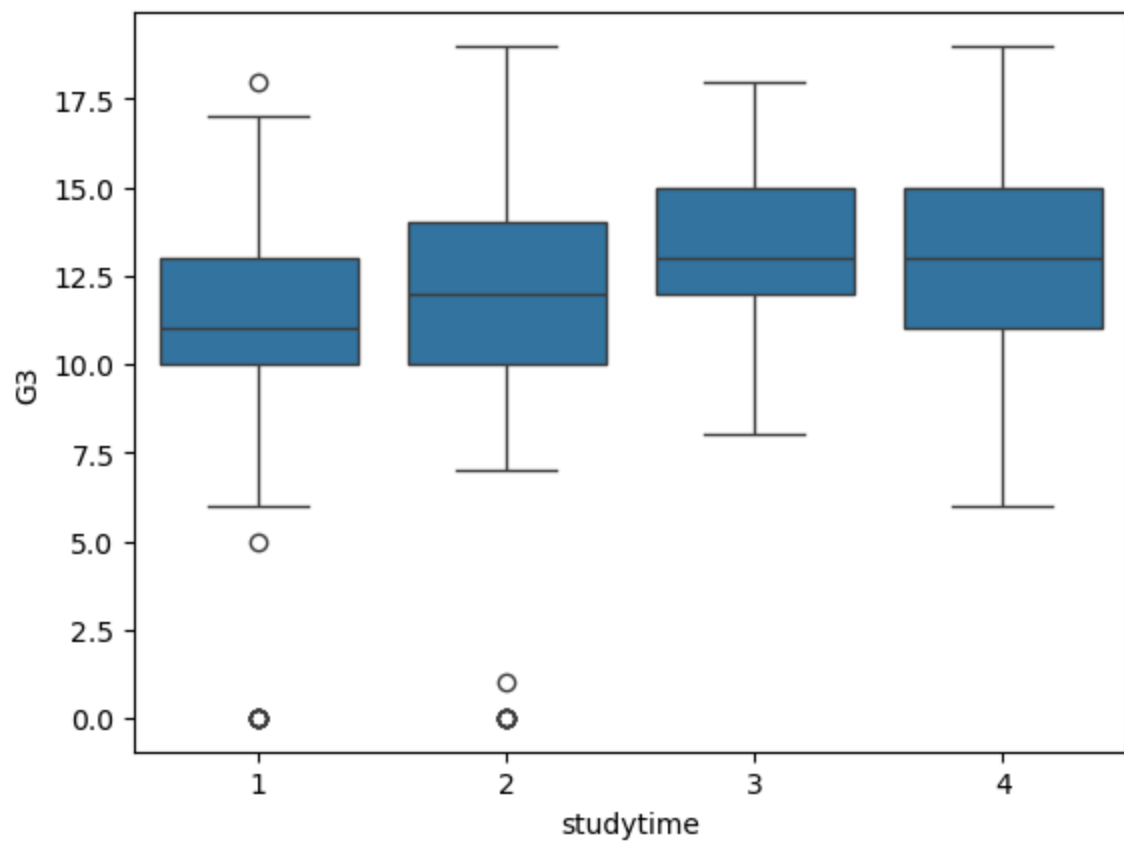
```
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
      'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',
      'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
      'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc',
      'Walc', 'health', 'absences', 'G1', 'G2'],
      dtype='object')
```

```
In [6]: import seaborn as sns
import matplotlib.pyplot as plt

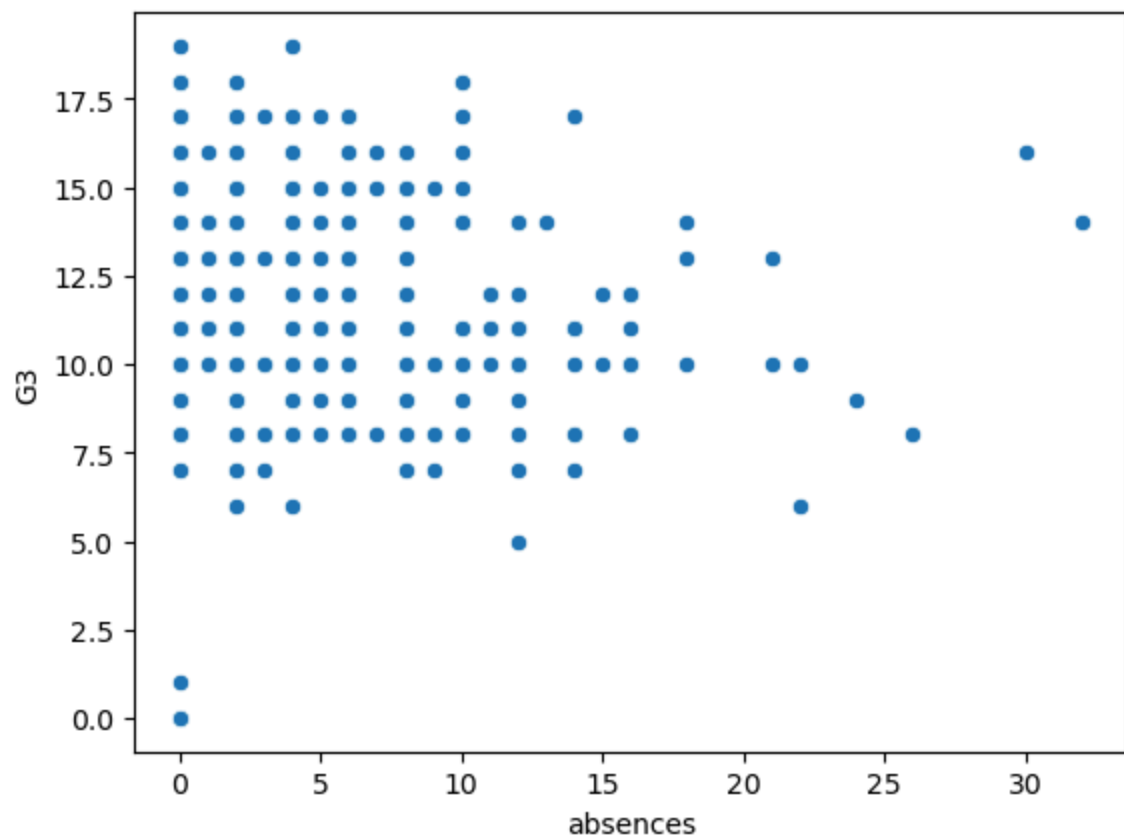
sns.histplot(df['G3'], bins=20)
plt.title("Distribution of Final Grades")
plt.show()
```



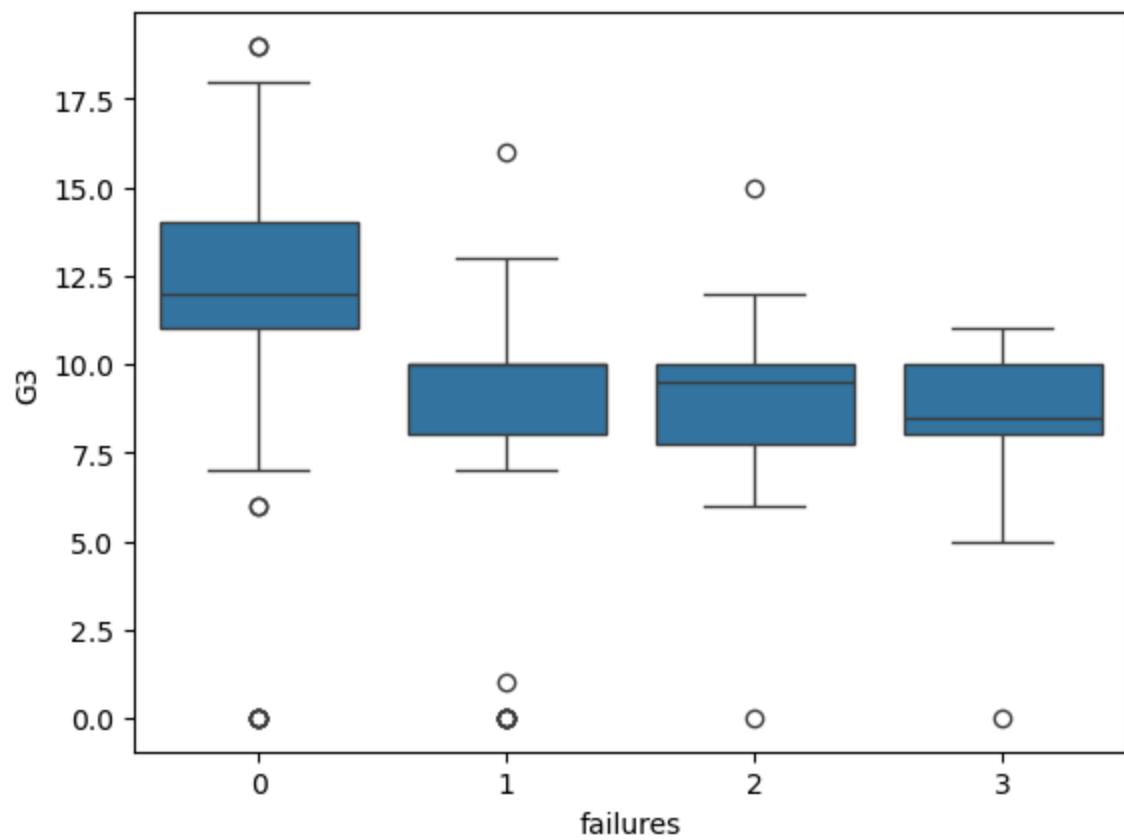
```
In [7]: sns.boxplot(x='studytime', y='G3', data=df)
plt.show()
```



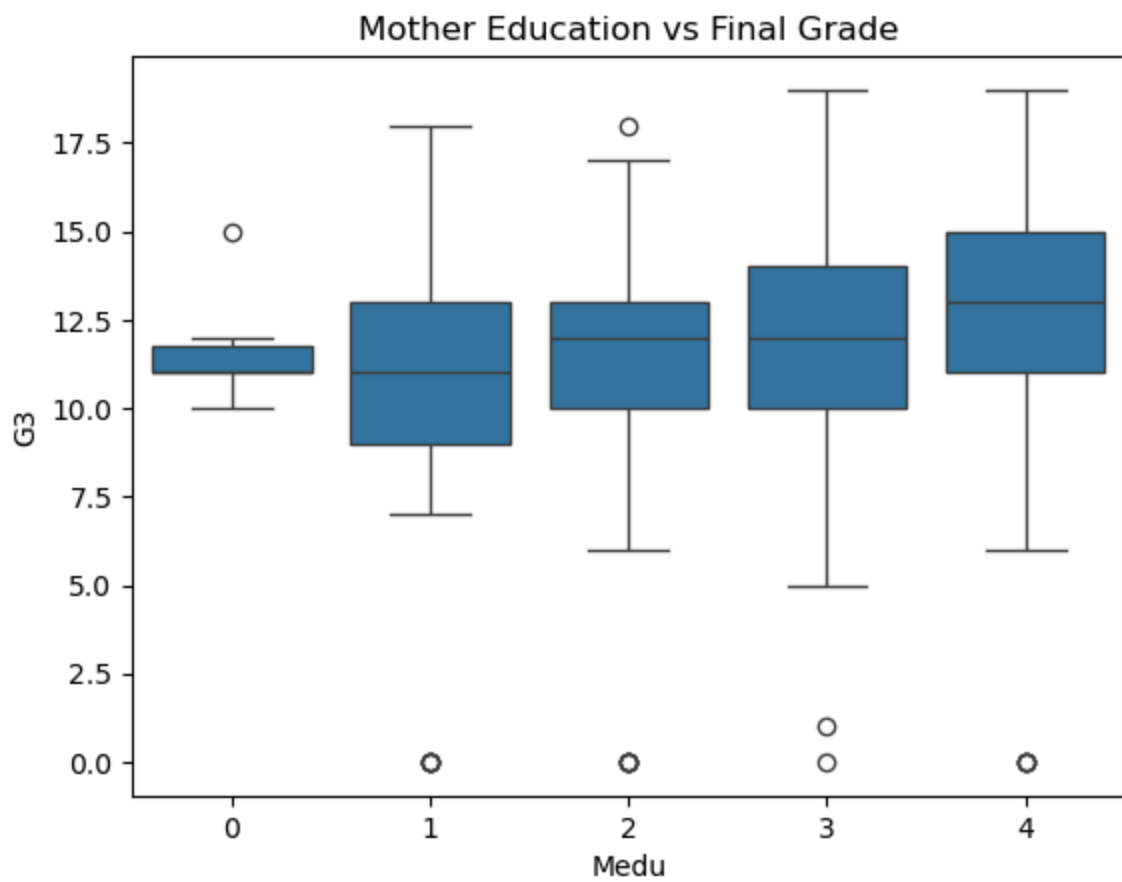
```
In [8]: sns.scatterplot(x='absences', y='G3', data=df)
plt.show()
```



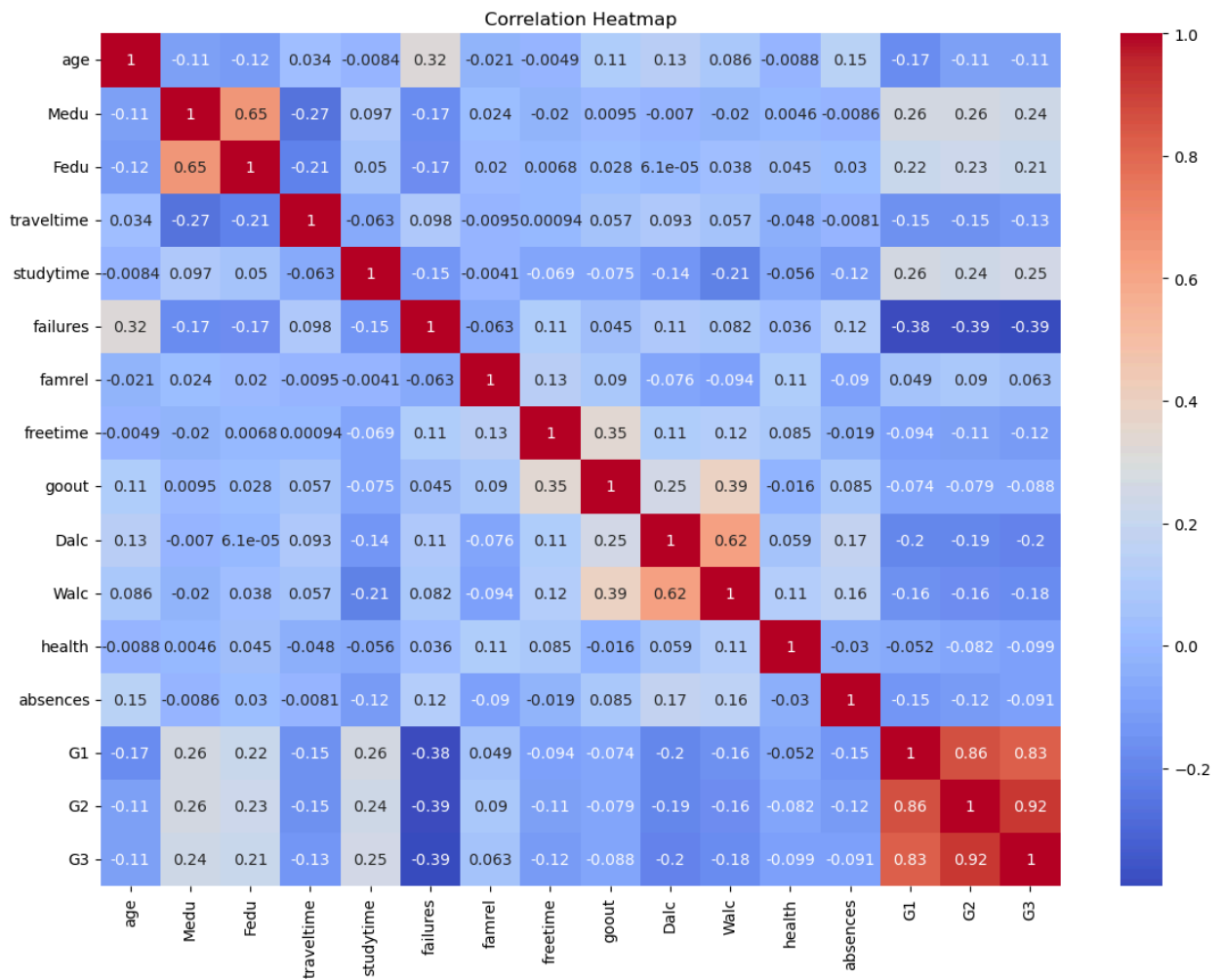
```
In [9]: sns.boxplot(x='failures', y='G3', data=df)
plt.show()
```



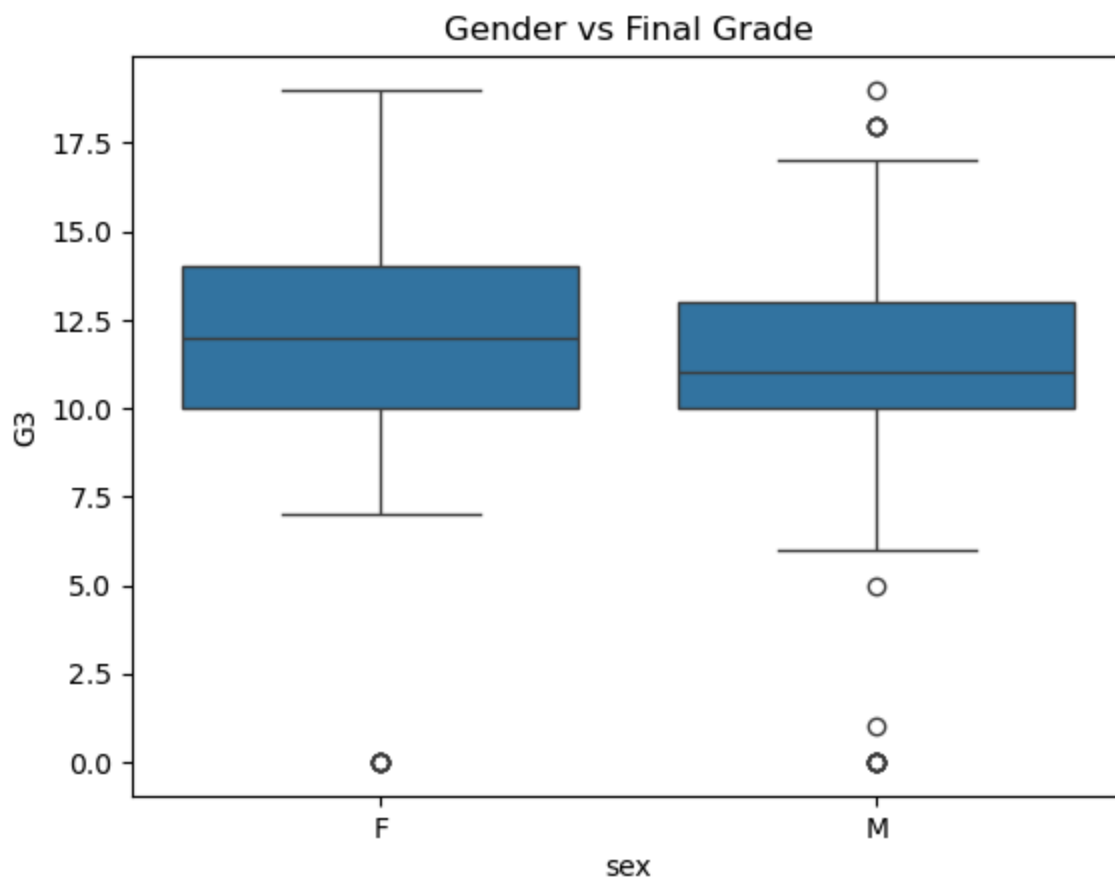
```
In [10]: sns.boxplot(x='Medu', y='G3', data=df)
plt.title("Mother Education vs Final Grade")
plt.show()
```



```
In [11]: numeric_df = df.select_dtypes(include=['int64'])
plt.figure(figsize=(14,10))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



```
In [12]: sns.boxplot(x='sex', y='G3', data=df)
plt.title("Gender vs Final Grade")
plt.show()
```



```
In [13]: from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
print(df.head())
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	\
0	0	0	18	1	0	0	4	4	0	4	...	
1	0	0	17	1	0	1	1	1	0	2	...	
2	0	0	15	1	1	1	1	1	0	2	...	
3	0	0	15	1	0	1	4	2	1	3	...	
4	0	0	16	1	0	1	3	3	2	2	...	

	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3	Performance
0	3	4	1	1	3	4	0	11	11	0
1	3	3	1	1	3	2	9	11	11	0
2	3	2	2	3	3	6	12	13	12	0
3	2	2	1	1	5	0	14	14	14	1
4	3	2	1	2	5	0	11	13	13	0

[5 rows x 34 columns]

```
In [14]: selected_features = [
```



```

'school',
'sex',
'age',
'address',
'famsize',
'Pstatus',
'Medu',
'Fedu',
'Mjob',
'Fjob',
'guardian',
'studytime',
'failures',
'schoolsup',
'famsup',
'paid',
'higher',
'internet',
'absences',
'G1',
'G2'
]
X = df[selected_features]

y = df['Performance']

print("\nFEATURES:\n")
print(X.columns)

```

FEATURES:

```

Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
      'Mjob', 'Fjob', 'guardian', 'studytime', 'failures', 'schoolsup',
      'famsup', 'paid', 'higher', 'internet', 'absences', 'G1', 'G2'],
      dtype='object')

```

```

In [15]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_s

```

```

In [16]: from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train_Scaled=scaler.fit_transform(X_train)
X_test_Scaled=scaler.transform(X_test)

```

```

In [20]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,classification_report,confusion_mat
lr = LogisticRegression(max_iter=2000,class_weight='balanced')
lr.fit(X_train_Scaled, y_train)
lr_pred=lr.predict(X_test_Scaled)
lr_accuracy = accuracy_score(y_test, lr_pred)
print("LOGISTIC REGRESSION RESULTS")
print("\nAccuracy :", lr_accuracy)

```

```

print("\nClassification Report:\n")
print(classification_report(y_test, lr_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, lr_pred))

```

LOGISTIC REGRESSION RESULTS

Accuracy : 0.8307692307692308

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.76	0.83	71
1	0.86	0.95	0.90	44
2	0.55	0.80	0.65	15
accuracy			0.83	130
macro avg	0.77	0.84	0.79	130
weighted avg	0.85	0.83	0.83	130

Confusion Matrix:

```

[[54  7 10]
 [ 2 42  0]
 [ 3  0 12]]

```

```

In [21]: from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=500, random_state=42, max_depth=10, clas
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
print("RANDOM FOREST RESULTS")
print("\nAccuracy :", rf_accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, rf_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, rf_pred))

```

RANDOM FOREST RESULTS

Accuracy : 0.8538461538461538

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.89	0.87	71
1	0.95	0.86	0.90	44
2	0.62	0.67	0.65	15
accuracy			0.85	130
macro avg	0.81	0.81	0.81	130
weighted avg	0.86	0.85	0.86	130

Confusion Matrix:

```
[[63  2  6]
 [ 6 38  0]
 [ 5  0 10]]
```

```
In [22]: importance = pd.DataFrame({
        'Feature': X.columns,
        'Importance': rf.feature_importances_
    })

importance = importance.sort_values(
    by='Importance',
    ascending=False
)
print("FEATURE IMPORTANCE")
print(importance)
```

FEATURE IMPORTANCE

	Feature	Importance
20	G2	0.358060
19	G1	0.255860
12	failures	0.052364
0	school	0.039104
18	absences	0.035643
2	age	0.029369
6	Medu	0.025647
7	Fedu	0.025390
8	Mjob	0.024145
11	studytime	0.023802
16	higher	0.020889
9	Fjob	0.020504
10	guardian	0.013707
13	schoolsup	0.013408
1	sex	0.011068
4	famsize	0.010207
14	famsup	0.009866
3	address	0.009559
17	internet	0.008921
5	Pstatus	0.006474
15	paid	0.006014